Fase 1: Análisis y Definición del Monolito

1.1 Lista Exhaustiva de Funcionalidades

Para que una tienda en línea sea funcional y comercialmente viable, el sistema debe contemplar las siguientes capacidades operativas:

- **Gestión de Usuarios (Identidad y Accesos)**
  - Registro de nuevos clientes y administradores.
  - Autenticación y gestión de sesiones (Login/Logout).
  - Administración de perfiles (datos personales, métodos de pago guardados).
  - Gestión de libreta de direcciones (múltiples direcciones de envío y facturación).
  - Recuperación y actualización de contraseñas.
- **Catálogo y Gestión de Productos**
  - Búsqueda de productos por palabras clave y términos relacionados.
  - Filtros dinámicos (rango de precio, marca, categoría, disponibilidad).
  - Visualización de detalles del producto (galería de imágenes, descripciones largas, especificaciones técnicas).
  - Control y actualización de inventario (stock disponible en tiempo real).
  - Administración de categorías, subcategorías y etiquetas.
- **Carrito de Compras**
  - Agregar, editar y eliminar productos del carrito.
  - Cálculo dinámico de subtotales, impuestos (IVA) y descuentos.
  - Aplicación y validación de cupones promocionales.
  - Persistencia del carrito temporal (mantener productos si el usuario cierra el navegador).
- **Gestión de Órdenes y Pagos**
  - Flujo de finalización de compra (*Checkout*).
  - Procesamiento de pagos con tarjeta o pasarelas de terceros (PayPal, etc).
  - Generación de número de orden y confirmación criptográfica de la transacción.
  - Historial de pedidos del usuario para consultas futuras.
  - Emisión de recibos electrónicos y comprobantes de compra.
- **Logística y Envíos**
  - Cálculo de tarifas de envío basadas en códigos postales, peso o volumen.
  - Asignación de paqueterías y generación de guías de rastreo.
  - Actualización del estado del pedido (empacando, en tránsito, entregado, devuelto).
- **Interacción y Soporte**
  - Sistema de reseñas y calificaciones (1 a 5 estrellas) con validación de "comprador verificado".
  - Listas de deseos (*Wishlists*) compartibles.
  - Envío de notificaciones automáticas (emails transaccionales por compra exitosa o envíos).

1.2 Agrupación Lógica (Módulos del Monolito)

Para estructurar la arquitectura monolítica, estas decenas de funcionalidades se agrupan en grandes dominios de negocio o "módulos" dentro del mismo código base:

1. **Módulo de Cuentas (Accounts):** Centraliza la Gestión de Usuarios, direcciones y seguridad.
2. **Módulo de Inventario (Inventory):** Administra el Catálogo, búsquedas, filtros y control de stock.
3. **Módulo de Transacciones (Transactions):** Unifica el Carrito de Compras, el procesamiento de Órdenes y los Pagos.
4. **Módulo de Despacho (Fulfillment):** Encapsula la logística, cálculo de tarifas de envío y actualizaciones de estado.
5. **Módulo de Fidelización (Engagement):** Contiene las Reseñas, Wishlists y disparadores de Notificaciones.

1.3 Bosquejo de la Aplicación Monolítica

En esta etapa de diseño, la tienda en línea opera bajo un **patrón arquitectónico monolítico fuertemente acoplado**. Sus características principales son:

- **Código Base Único:** Todos los módulos mencionados (Cuentas, Inventario, Transacciones, etc.) residen en un único repositorio y se ejecutan dentro del mismo proceso del servidor. Si se requiere modificar la lógica de los cupones de descuento, se debe compilar, detener y desplegar la aplicación entera.
- **Base de Datos Centralizada:** Existe una única base de datos relacional (por ejemplo, PostgreSQL o MySQL) que almacena la totalidad de la información. Las tablas están rígidamente vinculadas mediante restricciones de integridad referencial (*Foreign Keys*). Por ejemplo, la tabla `Orders` tiene un acoplamiento a nivel base de datos directo con las tablas `Users` y `Products`.
- **Flujo de Ejecución Secuencial:** Las operaciones complejas ocurren en un solo hilo o bloque sincrónico. Al presionar "Pagar", el monolito descuenta el stock invocando funciones del Módulo de Inventario, procesa el cobro y guarda la compra en la base de datos de manera atómica.
- **Limitaciones (El motivo de la transición):** Este diseño presenta cuellos de botella para el escalamiento. Si un evento masivo aumenta las búsquedas de productos de forma drástica, no es posible asignar más recursos computacionales únicamente al Módulo de Inventario; se debe instanciar y replicar todo el monolito, desperdiciando recursos. Además, un fallo de memoria en el Módulo de Despacho puede provocar la caída total del sistema, impidiendo que otros usuarios sigan navegando por el catálogo.

Fase 2: Estrategia de Microservicios y Responsabilidades

Para resolver los problemas de escalabilidad y acoplamiento del monolito, el sistema se divide en servicios independientes. Cada microservicio es dueño de su propio dominio de negocio y se puede desplegar, escalar y mantener sin afectar a los demás.

2.1 Lista de Microservicios

El monolito se ha fragmentado en los siguientes 7 microservicios principales:

1. **User Service** (Servicio de Usuarios)
2. **Catalog Service** (Servicio de Catálogo)
3. **Inventory Service** (Servicio de Inventario)
4. **Cart Service** (Servicio de Carrito)
5. **Order Service** (Servicio de Órdenes)
6. **Payment Service** (Servicio de Pagos)
7. **Shipping Service** (Servicio de Envíos)

2.2 Lista de Responsabilidades por Servicio

A continuación, se define estrictamente el alcance de cada microservicio (lo que hace y lo que no hace), garantizando la cohesión y evitando la duplicidad de lógica.

- **User Service**
  - *Responsabilidades:* Gestiona el registro, autenticación, perfiles de usuario, roles (cliente vs. admin) y libretas de direcciones.
  - *Límites:* No guarda el historial de compras, solo provee la identidad de quién está comprando.
- **Catalog Service**
  - *Responsabilidades:* Administra la información estática de los productos (nombres, descripciones, imágenes, categorías, precios base). Maneja las búsquedas y filtros del frontend.
  - *Límites:* No sabe cuántos artículos quedan en el almacén; eso es responsabilidad del Inventario.
- **Inventory Service**
  - *Responsabilidades:* Mantiene el conteo exacto de las existencias (*stock*) en tiempo real. Se encarga de "apartar" y "descontar" unidades de forma segura cuando ocurre una compra.
  - *Límites:* No procesa cobros ni interactúa con el catálogo para modificar descripciones.
- **Cart Service**
  - *Responsabilidades:* Almacena el estado temporal de los carritos de compra de los usuarios activos. Aplica la lógica de cálculo de subtotales.
  - *Límites:* No genera comprobantes fiscales ni guarda el carrito permanentemente si el usuario no finaliza la compra.
- **Order Service**
  - *Responsabilidades:* Es el orquestador principal de la compra. Recibe el carrito, crea el registro de la orden, calcula el total final (con impuestos y envíos) y coordina con los demás servicios para completar el flujo.

    - *Límites:* No interactúa directamente con los bancos ni con las paqueterías.
- **Payment Service**
    - *Responsabilidades:* Se comunica con las pasarelas de pago externas (Stripe, PayPal, bancos). Genera tokens de transacción, procesa reembolsos y reporta si un pago fue aprobado o rechazado.
    - *Límites:* No sabe qué productos se compraron, solo conoce el monto a cobrar y el identificador de la orden.
- **Shipping Service**
    - *Responsabilidades:* Calcula las tarifas de envío, genera números de guía y actualiza el estado logístico del paquete (en tránsito, entregado).
    - *Límites:* No factura ni modifica el catálogo de productos.

2.3 Estrategia de Persistencia (Database per Service)

Para garantizar un verdadero desacoplamiento, aplicaremos el patrón **Database per Service** (Base de datos por servicio). Ningún servicio puede consultar la base de datos de otro de forma directa; deben comunicarse mediante APIs. Las bases de datos se han asignado según la naturaleza de los datos:

Microservicio	**Tecnología de Base de Datos**	**Justificación Técnica**
User Service	PostgreSQL (Relacional)	Requiere alta consistencia (ACID) y esquemas rígidos para manejar credenciales, relaciones de direcciones y roles de seguridad.
Catalog Service	MongoDB / ElasticSearch (NoSQL)	Requiere alta velocidad de lectura, búsquedas de texto completo y flexibilidad para almacenar productos con diferentes atributos (ej. ropa vs. electrónicos).
Inventory Service	PostgreSQL (Relacional)	Es crítico evitar condiciones de carrera (*race conditions*) al descontar inventario. Requiere transacciones ACID estrictas y bloqueos a nivel de fila.

Cart Service	Redis (En memoria / Key-Value)	Los datos del carrito son efímeros y requieren lecturas/escrituras extremadamente rápidas. Se configuran con un tiempo de vida (TTL) para que expiren automáticamente.
Order Service	PostgreSQL (Relacional)	El historial de compras y los estados financieros requieren integridad referencial estricta y auditoría financiera.
Payment Service	PostgreSQL (Relacional)	Almacena *logs* de transacciones, tokens de reembolso y estados de cobro con precisión absoluta.
Shipping Service	PostgreSQL o MySQL (Relacional)	Mantiene el rastreo de guías, catálogos de códigos postales y estados de envío.

Fase 3: Comunicación e Integración de Servicios

En una arquitectura de microservicios, el sistema funciona como una red de componentes que colaboran entre sí. Para que esta orquestación sea exitosa, se deben definir claramente los canales de comunicación y los mecanismos para manejar fallos.

3.1 Entradas y Salidas de cada Servicio

Cada microservicio expone interfaces claras. A continuación, se definen los datos generales que cada servicio espera recibir (Entrada) y lo que debe responder (Salida):

- **User Service**
  - *Entradas:* Credenciales de inicio de sesión (email/password), datos de registro, tokens de validación.
  - *Salidas:* Tokens de acceso (JWT), perfil del usuario, lista de direcciones guardadas.
- **Catalog Service**
  - *Entradas:* Términos de búsqueda, identificadores de categorías, filtros, ID de un producto.
  - *Salidas:* Detalles del producto, listas paginadas de resultados, metadatos.
- **Inventory Service**
  - *Entradas:* ID del producto, cantidad a verificar, cantidad a apartar o descontar.
  - *Salidas:* Nivel de stock actual, confirmación de reserva exitosa o error por falta de inventario.
- **Cart Service**
  - *Entradas:* ID de usuario/sesión, ID del producto a agregar, cantidad.
  - *Salidas:* Estado actual del carrito, cálculo actualizado del subtotal y los impuestos.
- **Order Service**
  - *Entradas:* ID de usuario, artículos del carrito, ID de dirección de envío, método de pago.
  - *Salidas:* ID de la orden generada, estado de la orden (pendiente, pagada, enviada), total final a pagar.
- **Payment Service**
  - *Entradas:* ID de la orden, monto a cobrar, token de la tarjeta de crédito o cuenta de terceros.
  - *Salidas:* ID de transacción del banco, estado del pago (Aprobado/Rechazado), recibo de pago.
- **Shipping Service**
  - *Entradas:* Dirección de origen y destino, peso/volumen del paquete, ID de la orden.
  - *Salidas:* Costo del envío, número de guía (tracking), actualización de estado de entrega.

3.2 Comunicación Interna de los Servicios

Para evitar cuellos de botella, la comunicación backend-to-backend se divide en dos enfoques:

1. **Comunicación Síncrona (REST/gRPC):** Se usa para consultas inmediatas donde el sistema no puede avanzar sin la respuesta.
    - *Ejemplo:* El `Order Service` debe preguntarle al `Inventory Service` si hay stock disponible antes de intentar cobrar. Si no hay stock, el flujo se detiene inmediatamente.
2. **Comunicación Asíncrona (Colas de Mensajes/Eventos):** Utilizando un *Message Broker* (como RabbitMQ o Apache Kafka). Se usa para acciones que no necesitan bloquear al usuario.
    - *Ejemplo:* Una vez que el `Payment Service` confirma el cobro, emite un evento *"Pago_Exitoso"*. El `Shipping Service` escucha este evento en segundo plano y comienza a generar la guía de envío sin hacer esperar al usuario en la pantalla de confirmación.

3.3 Patrón API Gateway (Backend for Frontend)

Los clientes (navegador web o aplicación móvil) **no deben** comunicarse directamente con los microservicios, ya que esto expondría la arquitectura interna y generaría múltiples peticiones desde el cliente. Para solucionarlo, se implementa un **API Gateway**:

- **Función:** Actúa como la única puerta de entrada al sistema. Recibe todas las peticiones web/móviles.
- **Enrutamiento:** Sabe a qué microservicio mandar cada petición.
- **Validación de Seguridad:** Es el encargado de interceptar el token JWT de la petición, verificar que sea válido e inyectar el ID del usuario antes de pasar la petición a los microservicios internos.
- **Agregación:** Puede tomar una sola petición del frontend, consultar al `Order Service` y al `Shipping Service`, unir ambas respuestas, y devolver un solo paquete de datos al cliente.

3.4 Manejo de Transacciones Distribuidas

Dado que cada servicio tiene su propia base de datos, ya no podemos usar transacciones SQL tradicionales (bloqueos ACID) para garantizar que todo el proceso de compra sea atómico. Para resolver el problema de fallos parciales, utilizaremos el **Patrón Saga (Orquestación)** liderado por el `Order Service`.

Escenario de Compensación (Rollback Distribuido):

1. El usuario presiona "Comprar".
2. El `Order Service` crea la orden en estado *Pendiente*.
3. El `Order Service` le pide al `Inventory Service` que descuente 1 unidad (Exitoso).
4. El `Order Service` le pide al `Payment Service` que realice el cobro.

5. **Fallo:** El banco rechaza la tarjeta por fondos insuficientes.
6. **Acción de Compensación:** Como el pago falló, el `Order Service` lanza inmediatamente un comando de compensación al `Inventory Service` ordenando: *"Restaura 1 unidad de este producto, la venta se canceló"*.
7. El estado de la orden se marca como *Cancelada*, garantizando la consistencia de los datos sin bloquear las bases de datos.

Fase 4: Diseño de la Interfaz (GraphQL)

A diferencia de las APIs REST tradicionales que exponen múltiples URLs (ej. `/usuarios`, `/productos`), GraphQL centraliza la comunicación para evitar el *Over-fetching* (traer datos de más) y el *Under-fetching* (quedarse corto y hacer múltiples peticiones).

4.1 Endpoint y Método HTTP

- **Endpoint Único:** `/api/graphql`
- **Método HTTP:** **`POST`**. (Aunque GraphQL se usa para consultar datos, las consultas se envían en el "Body" de la petición HTTP, por lo que el estándar técnico es utilizar siempre el método POST para enviar tanto *Queries* como *Mutations*).
- **Autenticación:** Las peticiones protegidas incluyen un token en las cabeceras HTTP: `Authorization: Bearer <JWT_TOKEN>`. El API Gateway intercepta este token antes de procesar el GraphQL.

4.2 Definición de Operaciones Principales

En GraphQL, las operaciones se dividen estrictamente en dos categorías:

1. **Queries (Lectura / Consultas)**
    - `getProductDetails(id)`: Consulta al *Catalog Service* y al *Inventory Service* para mostrar la página de un producto con su stock actual.
    - `getUserProfile()`: Consulta al *User Service* para cargar los datos del cliente.
    - `getCart(userId)`: Consulta al *Cart Service* para obtener el subtotal.
2. **Mutations (Escritura / Modificaciones)**
    - `addToCart(productId, quantity)`: Modifica el estado en el *Cart Service*.
    - `processCheckout(cartId, paymentToken)`: Inicia la saga de compras orquestada por el *Order Service*.

4.3 Ejemplos Estructurales (JSON)

Ejemplo 1: Operación Query (Consultar un producto) El frontend solicita **únicamente** el nombre, precio y disponibilidad, ignorando las descripciones largas para ahorrar ancho de banda.

Entrada (Petición HTTP POST enviada al servidor):

```
{
  "query": "query GetProduct($id: ID!) { product(id: $id) { name price inStock } }",
  "variables": {
    "id": "PROD-8472"
  }
}
```

Salida (Respuesta HTTP 200 OK desde el servidor):


```
{
  "data": {
    "product": {
      "name": "Laptop Victus 15",
      "price": 18500.00,
      "inStock": true
    }
  }
}
```


Ejemplo 2: Operación Mutation (Finalizar la compra / Checkout) El frontend envía la instrucción de cobrar y crear el pedido.

Entrada (Petición HTTP POST enviada al servidor):

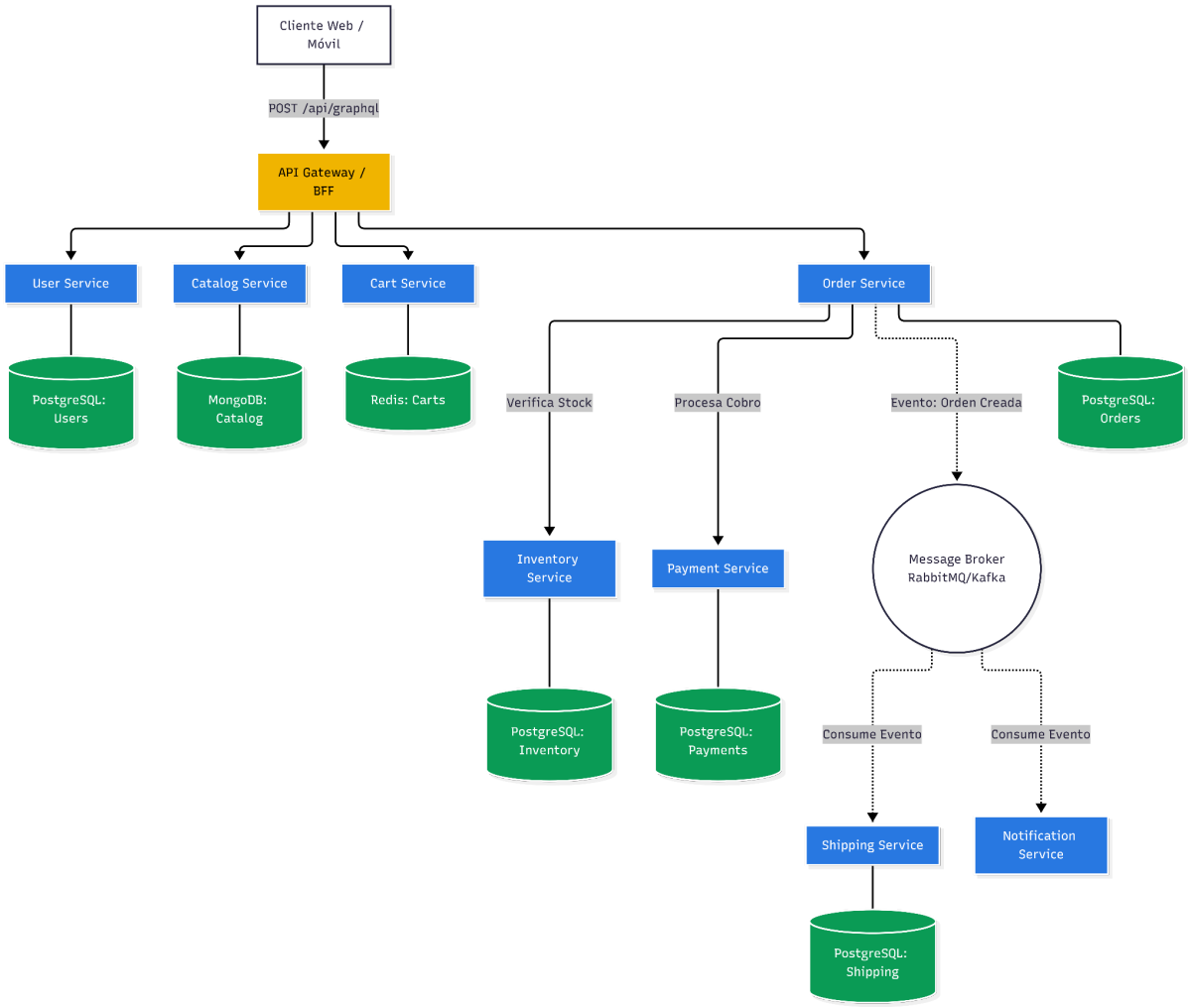
JSON


```
{
  "query": "mutation Checkout($cartId: ID!, $paymentToken: String!) {
processCheckout(cartId: $cartId, paymentToken: $paymentToken) { orderId status message
} }",
  "variables": {
    "cartId": "CART-551",
    "paymentToken": "tok_visa_4242"
  }
}
```


Salida (Respuesta HTTP 200 OK desde el servidor):


```
{
  "data": {
    "processCheckout": {
      "orderId": "ORD-2026-09",
      "status": "SUCCESS",
      "message": "Pago procesado correctamente, preparando envío."
    }
  }
}
```

Fase 5: Consolidación Visual (Diagrama de Arquitectura)



Pautas clave que este diagrama comunica a tu profesor:

1. **Aislamiento de persistencia:** Cada bloque azul (servicio) tiene su propio cilindro verde (base de datos), cumpliendo la regla de oro de los microservicios.
2. **Punto de entrada único:** El API Gateway es el único componente amarillo que recibe tráfico del exterior.
3. **Comunicación híbrida:** Las líneas sólidas muestran la comunicación síncrona necesaria para cobrar y verificar inventario en tiempo real, mientras que las líneas punteadas hacia el *Message Broker* demuestran un diseño asíncrono para el envío y las notificaciones, evitando bloqueos en el sistema.